

MediClaim Orchestrator

Explaining the Claim Workflow

A speaking script for Figure 3: the claim from upload to payment

Multi-Agent Cashless and Reimbursement Claim Processing

Read the 'Say this' boxes out loud

October 2026

1. First, how to read this diagram

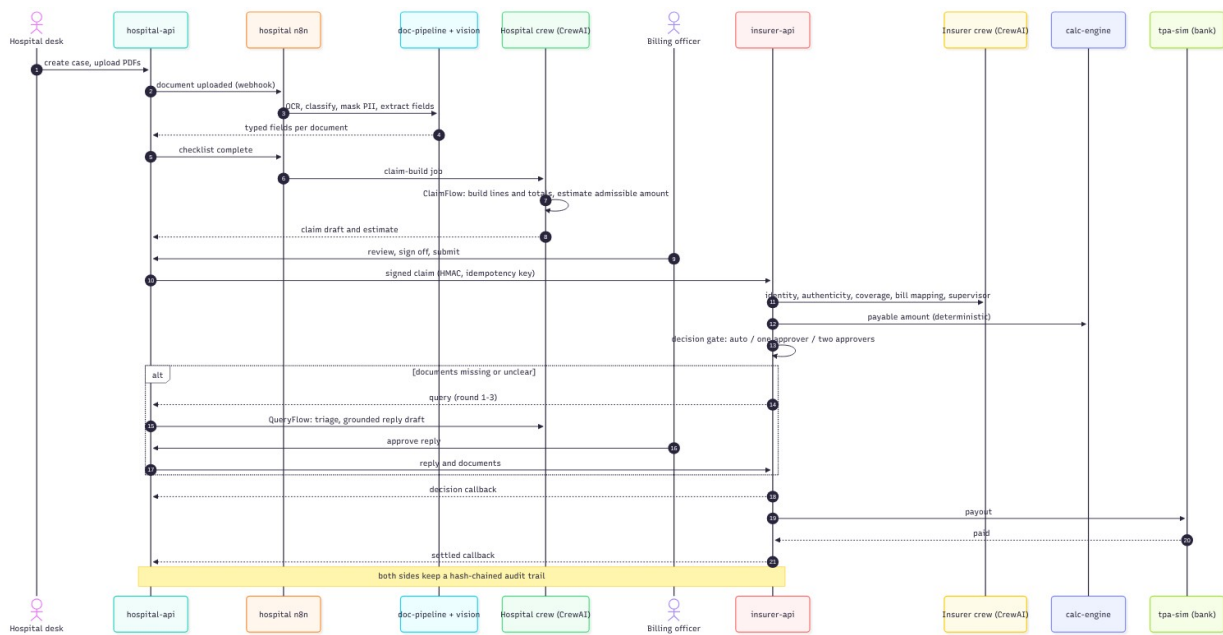


Figure 1: Figure 3 from the report: the claim from upload to settlement

This kind of picture is called a **sequence diagram**. It looks scary, but it's simple:

- The boxes at the top are the **players**. Each one is a person or a piece of software.
- The line going down from each box is that **player's timeline**.
- **Read from top to bottom**. Top is the start, bottom is the end.
- Each arrow is a **message** from one player to another. A solid arrow means 'please do this'. A dotted arrow means 'here is the answer'.
- The **black circles with numbers (1 to 21)** are the **steps in order**.
- The box marked '**alt**' only happens **sometimes**, when documents are missing or unclear.
- The **yellow bar at the bottom** means both sides keep a record of everything.

Say this

"This is a sequence diagram of one claim, from the moment the hospital uploads the bills to the moment the money is paid. The boxes on top are the players, time goes from top to bottom, and each arrow is one message. There are 21 steps, and I'll walk you through them."

2. The players and the tech behind each one

There are 10 players in the diagram. Two are people, eight are software. Here is who they are, what they're built with, and why.

Player	Who is it, in simple words	Tech stack used	Why this tech
Hospital desk	The person at the hospital who uploads the patient's documents	Web screen: Next.js, React, TypeScript	Simple, fast web screen that works in any browser
hospital-api	The hospital's main system. Keeps every case and every rule	Python, FastAPI, PostgreSQL, Redis, MinIO, Keycloak	FastAPI checks every request; Postgres keeps records safe; MinIO stores the files
hospital n8n	The hospital's manager. Starts each job at the right time	n8n (workflow tool, in Docker)	Good at waiting, retrying, reminders; you can see the process as a picture
doc-pipeline + vision	Reads the documents: OCR, finds fields, hides private data, checks photo quality	Python, Tesseract (OCR), Poppler (PDF), Presidio (privacy), OpenCV (images), ClamAV (virus scan)	Free, proven tools that run on our own laptop
Hospital crew (CrewAI)	The hospital's AI agents. Build the claim and estimate the payout	CrewAI, Ollama with gemma4, Qdrant for policy search	CrewAI is the agent framework the guidelines ask for; the model runs locally so data stays private
Billing officer	The hospital person who checks the claim and presses submit	Same web screen (Next.js)	A human always signs off before sending
insurer-api	The insurer's main system. Receives the claim, runs checks, decides	Python, FastAPI, PostgreSQL, Redis, Keycloak	Same reasons as hospital-api; kept fully separate because it's a different company
Insurer crew (CrewAI)	The insurer's AI agents. Check identity, bills, policy coverage	CrewAI, Ollama, Qdrant (RAG)	Agents read and explain; guardrails check their answers
calc-engine	The calculator. Works out exactly how much to pay	Plain Python with exact decimals	Money must be exact. No AI touches the money
tpa-sim (bank)	A pretend bank that sends the money	Python, FastAPI	Lets us show payment without a

Player	Who is it, in simple words	Tech stack used	Why this tech
			real bank

Holding it all together: Docker Compose starts the containers, uv installs the Python parts, and one command, `make demo`, runs everything.

Say this

"On the hospital side we have the desk staff, the hospital API, its n8n workflows, the document reader and the hospital's CrewAI agents. On the insurer side we have the insurer API, its CrewAI agents, a calculation engine and a bank simulator. The two sides are separate systems, just like two separate companies, and they only talk through signed messages."

3. Walking through the steps

Part A: The hospital gets the claim ready (steps 1 to 9)

Step 1: The desk uploads the documents

What happens: The hospital desk creates a case for the patient and uploads the PDFs: bills, discharge summary, ID card, policy card.

Say this

"It all starts at the hospital desk. The staff create a case for the patient and upload the documents, the bills, the discharge summary, the ID and the policy card."

Step 2: The API tells n8n

What happens: hospital-api saves the file and calls n8n through a webhook: 'a document was uploaded'.

Say this

"The hospital API saves the file and rings a bell for n8n, using a webhook. Basically it says, hey, a new document just came in."

Step 3: n8n sends it to be read

What happens: n8n sends the file to the document pipeline and the vision service. They do OCR, find what kind of document it is, hide personal data, and pull out the fields. It's also virus-scanned and checked for blur.

Say this

"n8n then sends the document to be read. The file is virus-scanned, checked for blur, and the text is read with OCR. The system figures out what type of document it is, picks out fields like name, dates and amounts, and hides personal details before any AI sees them."

Step 4: The fields come back

What happens: The document pipeline sends the typed fields back to hospital-api, one document at a time.

Say this

"The reader sends the fields back to the hospital API, neatly typed, for each document."

Step 5: The checklist is complete

What happens: When all required documents are in, hospital-api tells n8n: 'checklist complete'.

Say this

"Once every required document is there, the API marks the checklist as complete."

Step 6: Start building the claim

What happens: The claim-build job goes to the hospital CrewAI crew.

Say this

"Now the claim build starts, and this is where the CrewAI agents come in."

If asked: "In our code the API starts the crew directly, and n8n runs every 5 minutes as a safety net that restarts any build that got stuck."

Step 7: The agents build the claim and the estimate

What happens: Inside the crew, the ClaimFlow builds the bill lines and totals, and estimates the admissible amount, which is how much the insurer will probably pay.

Say this

"The hospital crew runs a CrewAI Flow called ClaimFlow. It puts every bill line into a category, adds up the totals, and then reads the patient's own policy wording to estimate how much the insurer will actually pay. The AI finds the rules, like the co-pay and the room rent limit, with the exact sentence as proof, and plain code does the maths."

Step 8: The draft comes back

What happens: The crew sends the claim draft and the estimate back to hospital-api.

Say this

"The agents hand back a draft claim and the estimate. The API then runs 12 checks on it, things like do the totals add up, do the dates match, is the name right."

Step 9: The billing officer signs off

What happens: The billing officer reviews the claim, fixes anything, signs off and presses submit.

Say this

"Then a real person, the billing officer, looks at it, can edit anything, sees the

estimate, and signs off. The AI never sends a claim by itself."

Part B: The claim goes to the insurer (steps 10 to 13)

Step 10: The signed claim is sent

What happens: hospital-api sends the claim to insurer-api, signed with HMAC and carrying an idempotency key.

Say this

"The claim is sent to the insurer. It's signed with an HMAC signature, so nobody can change it on the way, and it has an idempotency key, which is just a unique ID, so even if it gets sent twice it's only processed once."

Step 11: The insurer's agents check it

What happens: insurer-api asks the insurer CrewAI crew to check identity, authenticity, coverage, bill mapping, and to write a supervisor summary.

Say this

"On the insurer side, the CrewAI agents get to work. One checks identity, is this really the right patient. One checks authenticity, do the bills look genuine. One checks coverage, what does the policy say, and it quotes the clauses. One maps the bill lines, and a supervisor agent writes a short summary for the reviewer."

Step 12: The calculator works out the amount

What happens: insurer-api asks the calc-engine for the payable amount. This is deterministic: the same input always gives the same answer.

Say this

"Then the calculation engine works out the exact amount to pay. This is plain code with exact decimals, not AI, because money has to be exact and explainable."

Step 13: The decision gate

What happens: insurer-api decides who approves: the system itself, one approver, or two approvers.

Say this

"Next is the decision gate. If the claim is clean and up to fifty thousand rupees, the system can approve it automatically. Above that, one person approves. Above five lakh, two people approve."

"And if any check raised a warning, it always goes to a person, however small it is. In our tests on 300 claims, the system never wrongly auto-approved a claim."

Part C: Only if something is missing (steps 14 to 17, the 'alt' box)

This part only happens when documents are missing or unclear. It can repeat up to 3 times.

Step 14: The insurer sends a query

What happens: insurer-api sends a query to hospital-api (round 1, 2 or 3).

Say this

"Now, the box marked 'alt' only happens sometimes. If something is missing or unclear, the insurer sends a query back to the hospital. This can happen up to three rounds. After that, it goes to a senior reviewer."

Step 15: The hospital's agents sort it and draft a reply

What happens: hospital-api starts the hospital crew's QueryFlow. It triages the query and writes a grounded reply draft.

Say this

"The hospital crew runs another flow called QueryFlow. First it sorts the query, do we need to send a document, or just explain something? Then it drafts a reply using only real facts from the case. That's what 'grounded' means, no made-up information."

Step 16: The billing officer approves the reply

What happens: The billing officer reads the draft and approves it.

Say this

"Again, a person checks the reply before it goes."

Step 17: The reply goes back

What happens: hospital-api sends the reply and the documents to insurer-api, and the checks run again.

Say this

"The reply and any extra documents go back to the insurer, and the checks run again."

Part D: Decision and payment (steps 18 to 21)

Step 18: The decision goes back to the hospital

What happens: insurer-api sends the decision callback to hospital-api: approved, partly approved, or rejected.

Say this

"Once there's a decision, the insurer sends it back to the hospital straight away."

Step 19: Payment is sent

What happens: insurer-api asks the bank simulator to pay out.

Say this

"If it's approved, the insurer tells the bank to pay. In our project the bank is a simulator."

Step 20: The bank confirms

What happens: tpa-sim says 'paid'.

Say this

"The bank confirms the money is paid."

Step 21: The hospital is told it's settled

What happens: insurer-api sends the 'settled' callback to hospital-api, and the case is closed.

Say this

"And finally the hospital gets a 'settled' message, and the case is closed."

The yellow bar: the audit trail

Say this

"The yellow bar at the bottom means both sides keep an audit trail of every single step. Each record is chained to the one before it with a hash, a bit like a blockchain, so if anyone secretly changes an old record, the chain breaks and we can tell."

4. The full script in one go (about 2 minutes)

If you just need to explain the whole diagram quickly, read this:

Say this

"This diagram shows one claim, from upload to payment. Time goes from top to bottom, and each arrow is a message.

It starts at the hospital desk, where the staff upload the patient's documents. The hospital API saves them and triggers an n8n workflow, which sends each file to be virus-scanned, read with OCR, classified, and have personal data hidden.

When all documents are in, the hospital's CrewAI agents build the claim. They sort the bill lines, add up the totals, and read the patient's policy to estimate how much the insurer will pay. The billing officer checks it and submits.

The claim goes to the insurer signed, so it can't be changed. There, the insurer's CrewAI agents check identity, authenticity and coverage, and the calculation engine works out the exact amount. Then the decision gate decides: small clean claims are approved automatically, bigger ones go to one or two people.

If something is missing, the insurer sends a query. The hospital's agents sort it and draft a reply, and the officer approves it before it goes back.

Finally, the decision goes back to the hospital, the bank pays, and the hospital gets a 'settled' message. Both sides keep a tamper-proof audit trail of every step.

So in short: AI reads and writes, code does the rules and money, and people approve the important decisions."

5. Questions you may get about this diagram

Q: Where exactly is the AI in this diagram?

In two places: the Hospital crew (steps 7 and 15) and the Insurer crew (step 11). Everything else is normal code or people.

Q: Why is the calc-engine separate from the agents?

Because money must be exact. AI can make small mistakes with numbers, so we never let it calculate the amount.

Q: What does n8n do here?

It's the manager. It reacts to events like 'document uploaded', sends work to the right service, waits, retries and reminds. It never makes the decision itself.

Q: What is HMAC and why is it on step 10?

It's a digital signature using a secret key. The insurer can check the claim really came from this hospital and wasn't changed on the way.

Q: What is an idempotency key?

A unique ID on each message. If the same message arrives twice because of a network retry, it's only handled once.

Q: What happens if an agent fails?

That step is marked as 'degraded', the claim keeps going, and it goes to a person to decide. Nothing breaks.

Q: Why are there two people in the diagram?

Because humans stay in control. The billing officer approves what the hospital sends, and approvers on the insurer side decide anything big or doubtful.